

Roteiro - Câmera no Godot e Cutscene.

E aí, galerinha! Beleza? Hoje vamos fazer uma introdução completa ao sistema de câmeras no Godot.

As câmeras são um dos elementos mais importantes de qualquer jogo. Uma câmera bem configurada pode mudar completamente a experiência do jogador, ajudando a criar imersão e fazendo com que ele realmente se sinta dentro do personagem e do mundo do jogo.

Neste tutorial, vamos entender o básico de como funcionam as câmeras no Godot e aprender a configurá-las de forma simples para começar a utilizá-las nos seus projetos.

Para entender melhor, vamos dividir o conteúdo em 5 partes: Câmeras 2D, Câmeras 3D, Transições, Cutscenes e até como fazer Câmeras de Segurança (e Portais).

Então, bora começar!

Para entender melhor o sistema de câmeras no Godot, vamos dividir o conteúdo nos seguintes tópicos:

1 - Câmeras 2D

A Câmera 2D é extremamente simples de se usar. Basicamente, se você colocá-la como "filha" de um nó de física (seja ele um `CharacterBody2D`, `RigidBody2D` ou até um node estático), ela já vai automaticamente se linkar à posição daquele nó, sem precisar de código complexo. Para onde o personagem for, a câmera vai atrás.

Mas o grande poder dela está nas opções que ficam ali no painel do Inspetor:

Zoom: É exatamente o que o nome diz, mas preste atenção na matemática do Godot! O padrão é X: 1, Y: 1. Se você aumentar esse número para 2, a imagem vai ficar duas vezes maior (ou seja, a câmera dá um zoom in e chega perto do personagem). Se você diminuir para 0.5, as coisas ficam menores na tela, mostrando mais do cenário (zoom out).

Offset (Deslocamento): Por padrão, a câmera coloca o seu personagem no exato centro da tela. Mas e se você estiver fazendo um jogo de corrida e quiser que o jogador veja mais do que está na frente dele do que atrás? Você altera os valores de Offset para "empurrar" o centro da câmera um pouco mais para a direita, esquerda, cima ou baixo, sem tirar ela de dentro do personagem.

Anchor Mode (Modo de Âncora): Isso aqui confunde muita gente no começo! O Godot te dá duas opções:

Drag Center (Padrão): A câmera foca no centro. Esse modo libera o uso das margens de arrasto (Drag Margins). Sabe quando o Mario anda um pouquinho pro lado e a tela não se move imediatamente, só começa a se mover quando ele chega perto da borda? Isso é o Drag Center trabalhando em conjunto com as margens.

Fixed TopLeft: Esse modo pega o canto superior esquerdo da sua tela e "gruda" exatamente onde o seu personagem está. É menos usado para seguir personagens, mas muito útil para posicionar mini-mapas ou interfaces.

Position Smoothing (Suavização): Uma dica de ouro! Se você ativar isso, a câmera deixa de seguir o jogador de forma dura e robótica, e passa a segui-lo como se estivesse presa por um elástico, criando um movimento muito mais profissional e suave.

1.1 - Alguns Tipos práticos de câmeras 2D são:

Plataforma: Usa muito o Drag Center e os limites (Limits) na aba do inspetor para impedir que o jogador veja o "vazio" fora do mapa.

Top Down: Visão de cima (estilo Pokémon antigo). Geralmente a câmera fica travada exatamente no centro do jogador, sem muita margem de arrasto.

Câmera Estática: Não segue o jogador. Ela fica parada na cena. Quando o jogador sai da tela, você desativa essa câmera e ativa a da próxima sala (muito clássico em jogos estilo Zelda).

2 - Câmeras 3D

Saindo do 2D e indo para o 3D, a brincadeira fica um pouco mais complexa, porque agora temos a profundidade (o eixo Z) e a rotação livre em todos os ângulos. No 3D, as opções do inspetor continuam em sua maioria iguais às do 2D, mas com algumas adições importantes para esse novo ambiente:

Cull Mask: Funciona como um filtro que define exatamente quais camadas (layers) do jogo a câmera consegue enxergar e quais ela deve ignorar.

Near / Far (Perto / Longe): Controla os limites da visão, definindo o quão perto um objeto precisa estar para ser "cortado" da tela e até qual distância máxima o horizonte será renderizado.

Current (Atual): É a opção que você marca para avisar à engine que esta é a câmera principal que deve transmitir a imagem para o jogador.

Além disso, temos uma diferença crucial: em vez de Zoom, no 3D nós lidamos com o FOV (Field of View - Campo de Visão). Um FOV maior deixa a tela parecendo uma "lente olho de peixe", dando a sensação de mais velocidade e mostrando mais detalhes nas laterais. Já um FOV menor achata a imagem e foca num ponto só, como se fosse uma lente de aproximação.

2.1 - Tipos de câmeras 3D

Câmera de Plataforma (Terceira Pessoa): Estilo Mario 64. A câmera orbita o personagem. No Godot, o grande truque aqui não é colocar a câmera colada no jogador, mas sim

pendurar ela num nó chamado SpringArm3D. Ele funciona como um "pau de selfie inteligente": se você encostar de costas numa parede, em vez da câmera atravessar a parede e mostrar o "limbo" do jogo, o SpringArm encolhe automaticamente e aproxima a câmera do personagem!

Câmera Top Down / Isométrica: Visão de cima para jogos estilo Diablo. A câmera fica travada no alto, olhando para baixo num ângulo fixo, apenas acompanhando o personagem andar no chão.

Câmera Estática: Clássica de Resident Evil antigo. Câmeras colocadas nas quinas das salas, focadas em um ponto específico do cenário. Quando o jogador entra num corredor, você troca de qual câmera está mandando a imagem para a tela.

Over the Shoulder: A câmera "no ombro", colada nas costas do personagem e levemente deslocada pro lado (usando o Offset que aprendemos!). Usada em The Last of Us e God of War modernos.

FPS (Primeira Pessoa): A câmera fica literalmente dentro da cabeça do personagem. Aqui, o movimento do mouse rotaciona diretamente a própria câmera para olhar em volta.

3 - Transição de Câmera

Sabe quando você encontra um chefão, o jogo tira o controle de você, a câmera viaja até a cara do monstro e depois volta pro seu personagem?

No Godot, só uma câmera pode renderizar a tela por vez (usamos a função `nome_da_camera.make_current()` para decidir qual é). Mas se você só trocar de uma câmera para a outra, a tela vai dar um "corte seco" de cinema.

Podemos, por exemplo, apenas pegar e fazer um Tween da nossa câmera principal direto para a posição que queremos. A ideia parece boa, mas, na prática, isso gera uma bola de neve de problemas.

Você vai esbarrar em dores de cabeça chatas, desde ter que calcular na mão como retornar a câmera para a posição original, até lidar com teleportes bizarros - já que a câmera vai tentar voltar para onde o jogador estava, e não para onde ele está agora. Outro problema clássico é a transição ser interrompida e a sua câmera ficar travada no meio do nada se o jogador sair da área correndo.

Além de tudo isso, fazer dessa forma não é nem um pouco modular. É zero reutilizável. Você acabaria tendo que copiar e colar o mesmo bloco de código gigantesco em vários scripts diferentes sempre que quisesse colocar uma câmera nova na fase.

(explico o código que eu fiz).

Como trocar entre diferentes câmeras durante o jogo e criar transições suaves entre elas.

4 - Cutscenes (Cenas Cinematográficas)

Agora, se você quer assumir o papel de diretor de cinema e criar cutscenes, o seu melhor amigo no Godot é o AnimationPlayer.

Você não precisa programar dezenas de linhas de código para isso. Você cria uma linha do tempo no AnimationPlayer e vai gravando "Keyframes" (pontos-chave). Você coloca a câmera ali na porta, aperta o botão de gravar. Arrasta a agulha da linha do tempo pra frente, move a câmera até o rosto do jogador e grava de novo. O próprio Godot vai calcular todo o movimento da câmera entre esses dois pontos de forma automática!

(tutorial de uma cutscene simples que eu ainda não fiz)

5 - Câmeras de Segurança e Portais

Isso aqui parece feitiçaria mas é tecnologia. E se você quiser criar uma tela de TV no cenário que mostra o que está acontecendo do outro lado da base? Ou um portal que mostra onde suas outras partes vão parar?

Para isso, usamos um nó chamado SubViewport. Você coloca uma câmera dentro desse SubViewport. A ideia é: a imagem que essa câmera captura não vai para a tela principal do jogador. Ela é transformada numa Textura (como se fosse uma imagem PNG animada em tempo real). Aí é só você pegar essa textura e aplicá-la no material da televisão ou no espelho do seu jogo!

(tutorial de um espelho simples que eu fiz)

(fazer portal 3)